

EShop Automation Testing Report

Student ID: 23127378 **Features:** FR-02 (Pool A), FR-11 (Pool B), FR-16 (Pool C)

1. Feature Selection

For this HW04, I have selected the same three features as in my HW02 AI Domain Testing assignment:

1. **FR-02:** Login and account logout (Pool A)
2. **FR-11:** Order history view (user) (Pool B)
3. **FR-16:** Product import from CSV (Pool C)

2. Automation Strategy (AI-First)

The automation scripts were entirely generated using a data-driven approach orchestrated by a custom-built AI Agent Skill (**tc-codegen**).

Process:

- **Data Parsing:** The original Markdown Domain Testing files from HW02 (**testcase/*.md**) were fed into the **tc-parse** skill to extract raw table data into structured JSON files (**e2e/test-data/*.data.json**).
- **Script Generation:** The **tc-codegen** skill ingested the JSON data and generated the Playwright test scripts. The scripts were built using Page Object Models (POM) for reusability.
- **Human Review & Fix:** After generating the scripts, I executed them and manually reviewed the results. I had to fix several timeout issues caused by the local Vite server crashing under heavy parallel load from Playwright (NS_ERROR_NET_EMPTY_RESPONSE).

3. Automation Implementation Details

- **Data-Driven Execution:** Test inputs and expected outputs are dynamically read from JSON arrays, iterating over categories like **positive**, **negative**, **boundary**, **security**, and **transaction**.
- **Assertions:** The scripts employ multiple robust assertion patterns:
 - Checking DOM visibility (**expect(locator).toBeVisible()**)
 - Checking text match (**expect(locator).toHaveText()**)
 - Validating CSS and attributes (**toHaveCSS()**, **toHaveAttribute()**)
 - Checking API responses (**expect(response.status()).toBe()**)
- **Multi-Browser:** Playwright is configured via **playwright.config.ts** to run across 3 projects: Chromium, Firefox, and WebKit simultaneously.

4. Gap Analysis & Review of AI Generation

What the AI missed during script generation:

1. **Complex UI Flows:** For FR-11, the AI struggled initially to construct complex XPath locators needed to locate a specific row in the order table based on multiple dynamic criteria (Order ID + Status). Human intervention was required to enforce simpler **.getByRole('row').filter(...)** logic.

2. **Asynchronous Race Conditions:** In FR-16 (CSV Import), the AI generated a script that triggered file upload *before* setting up the `page.waitForResponse()` listener, causing the script to occasionally miss the API response and timeout. The script was refined manually to attach the listener first.
3. **Flaky Error Capturing:** The AI initially did not use `.catch()` blocks for visibility assertions on negative test cases. Since the system contains intentional bugs, many error elements simply never appeared. Without `.catch()`, these tests crashed abruptly instead of elegantly logging the failure, hiding the true nature of the bugs.

5. Summary of Bug Findings

The automation successfully exposed 19 intentional bugs in the SUT, which were previously overlooked or manually tedious to verify in HW02:

- **FR-02:** 8 grouped bugs (Login attempts incremented by 2, Lockout time 180s, Missing semantic types, Invalid UI Error placement, Wrong labels, etc.)
- **FR-11:** 2 grouped bugs (Missing H1, Missing Empty State icon)
- **FR-16:** 9 grouped bugs (No Role Admin check, No Transaction Rollback, Naive CSV Parser splitting on quotes, Missing validations for price, name and category ID, Header failures).

Refer to *bug-report.md* for full defect details.